

Solución tarea 11

En esta práctica mezclamos varios modelos de IA (GPT, Gemini y Claude) con Gradio. Como yo no tengo ninguna de estas APIs de pago he añadido una llamada a Ollama, para comprobar que la interfaz realiza las diferentes funciones correctamente.

1. Imports

Primero debemos de importar diferentes librerías que nos harán falta para ejecutar el programa.

- Openai, anthropic y Google.generativeai: Son los import para hacer llamadas a las respectivas APIs
- Request la vamos a utilizar cuando ollama nos devuelva la solución
- Gradio para la interfaz gráfica
- Dotenv para cargar las API-Keys

```
from dotenv import load_dotenv
import os
import gradio as gr
import requests
from openai import OpenAI
import anthropic
import google.generativeai
```

En esta segunda parte cargamos el fichero .env donde tenemos puestas las claves de las APIs de las 3 IAs que vamos a utilizar.

```
#Cargamos el fichero .env y las variables de entorno
load_dotenv()
open_api_key = os.getenv("OPENAI_API_KEY")
anthropic_api_key = os.getenv("ANTHROPIC_API_KEY")
google_api_key = os.getenv("GOOGLE_API_KEY")
```

```
🔧 .env
1 OPENAI_API_KEY=
2 ANTHROPIC_API_KEY=
3 GOOGLE_API_KEY=
```

2. Comprotamiento de la IA

Posteriormente hemos creado una variable como prompt para las diferentes IAs

```
#Prompt que van a tener todas las IA
system_message = "Eres un asistente que ayudas a los usuarios a resolver sus dudas."
```

3. Funcion "controladora"

Esta función consiste en recibir la pregunta del usuario y el modelo del usuario, comprobará que IA quiere utilizar el usuario y la "llamará".

Las diferentes funciones consultar_*nombreIA* son las llamadas a las Apis de esa IA.

```
#Funcion que llama a la IA seleccionada
def procesar_pregunta(pregunta, modelo):
    if modelo == "ChatGPT":
        return consultar_gpt(pregunta)
    elif modelo == "Gemini":
        return consultar_gemini(pregunta)
    elif modelo == "Claude":
        return consultar_claude(pregunta)
    elif modelo == "Ollama":
        return consultar_ollama(pregunta)
```

4. Llamada a las IA

En cada captura se va a mostrar el método para llamar a una API y esta devuelve la respuesta.

- Llamada OpenAI

```
# ----- LLAMADA A OPENAI -----
def consultar_gpt(pregunta):
    # Protocolo de Llamada a la API de OpenAI
    openai = OpenAI()

    promts = [
        {"role": "system", "content": system_message},
        {"role": "user", "content": pregunta}
    ]

    # Realizamos la llamada
    completion = openai.chat.completions.create(
        model="gpt-3.5-turbo",
        messages=promts,
        temperature=0.2,
        max_tokens=50
    )

    # Extraemos y mostramos la respuesta
    respuesta = completion.choices[0].message.content
    return respuesta
```

- Llamada Gemini

```
# ----- LLAMADA A LA API DE GOOGLE (Gemini)-----
def consultar_gemini(pregunta):
    # Protocolo de Llamada a la API de Google
    google.generativeai.configure(api_key=google_api_key)
    gemini = google.generativeai.GenerativeModel(
        model_name="gemini-1.5-flash",
        system_instruction=system_message
    )

    # Definimos la temperatura del modelo
    configuracion = google.generativeai.types.GenerationConfig(
        temperature=0.9,
        max_output_tokens=100
    )

    # Realizamos la llamada
    response = gemini.generate_content(
        pregunta,
        generation_config=configuracion
    )

    return response.text
```

- Llamada Claude

```
# ----- LLAMADA A ANTHROPIC (Claude)-----
def consultar_claude(pregunta):
    # Conectamos con el cliente de Anthropic
    claude = anthropic.Anthropic()

    # Realizamos la llamada
    message = claude.messages.create(
        model="claude-3.5-sonnet-20240620",
        max_tokens=50,
        #max_tokens=200, Lo dejo puesto para que no tenga limite de tokens
        temperature=0.6,
        messages=[
            {"role": "system", "content": system_message},
            {"role": "user", "content": pregunta}
        ]
    )

    # Extraemos y mostramos la respuesta
    respuesta = message.content[0].text()
    return respuesta
```

- Llamada ollama

```

# ----- LLAMADA A OLLAMA -----
def consultar_ollama(pregunta):
    # URL por defecto de la API de Ollama
    OLLAMA_API_URL = "http://localhost:11434/api/generate"
    OLLAMA_MODEL = "llama3" # Asegúrate de que este modelo esté descargado

    # Combinamos los prompts para el formato de Ollama/llama
    full_prompt = f"Eres un asistente que va a resolver la siguiente pregunta: {pregunta}"

    # Configuramos la petición para limitar la longitud
    payload = {
        "model": OLLAMA_MODEL,
        "prompt": full_prompt,
        "stream": False,
        "options": {
            # El parámetro para limitar tokens de salida en Ollama es num_predict
            "num_predict": 100,
            "temperature": 0.9
        }
    }

    try:
        # Realizamos la llamada HTTP
        response = requests.post(OLLAMA_API_URL, json=payload)
        response.raise_for_status() # Manejo básico de errores HTTP

        # Extraemos y mostramos la respuesta
        data = response.json()
        respuesta = data.get("response", "Error al procesar la respuesta.")
        return respuesta

    except requests.exceptions.RequestException as e:
        return "Error al conectar con Ollama"

```

5. Ventanas gráfica

Por último, este bloque de código (utilizando Gradio) es una ventana gráfica, con un bloque de texto para que el usuario escriba su petición, un desplegable con las diferentes IAs, un botón de enviar, para enviar la petición al programa y un botón limpiar para eliminar todo el contenido y dejar la página restaurada.

```
with gr.Blocks(title="Comparador de IAs") as demo:

    gr.Markdown("## 🤖 Comparador de Modelos de IA")

    # 1. Entrada: Textbox multilínea
    pregunta = gr.Textbox(
        label="Pregunta",
        placeholder="Escribe aquí tu pregunta...",
        lines=5
    )

    # 2. Selector: Dropdown
    modelo = gr.Dropdown(
        choices=["ChatGPT", "Gemini", "Unknown", "Ollama"],
        value="Ollama", # Por defecto muestra Ollama
        label="Modelo de IA"
    )

    # 3. Salida: Markdown
    salida = gr.Markdown(label="Respuesta")

    # 4. Botones
    with gr.Row():
        btn_enviar = gr.Button("Enviar")
        btn_limpiar = gr.Button("Limpiar")

    btn_enviar.click(
        fn=procesar_pregunta,
        inputs=[pregunta, modelo],
        outputs=salida
    )
```

```
btn_limpiar.click(  
    fn=lambda: ("", "Ollama", ""),  
    outputs=[pregunta, modelo, salida]  
)  
  
demo.launch()
```

Para mostrar como quedaría:

 Comparador de Modelos de IA

Pregunta

2024 es un año bisiesto?

Modelo de IA

Ollama

¡Sí! El año 2024 es, en efecto, un año bisiesto. Los años bisiestos son aquellos que tienen 366 días, y ocurren cada 4 años (a excepción de los años que sean múltiplos de 100 pero no de 400).

Enviar

Limpiar